

FinTrack Pro

Project Documentation

Personal Finance Tracker | Browser-Based | No
Backend

1. What is FinTrack Pro?

FinTrack Pro is a personal finance tracking web app that runs entirely inside the browser. There is no backend server, no database setup, and nothing to install. Anyone can open it and immediately start tracking their income and expenses.

All data is saved in the browser so transactions stick around even after closing the tab. The app is built using plain HTML, CSS, and JavaScript - no frameworks, no build tools needed.

Core Idea

The whole app solves one simple problem: helping people see where their money goes. Every feature connects back to that goal - adding a transaction, watching the balance update instantly, filtering by type, or switching to a different currency symbol.

Features at a Glance

Feature	What it does
Add Transactions	Log income or expense with description, amount, date, and category
Live Balance	Current balance, total income, and total expense recalculate instantly
Cash Flow Chart	Bar chart gives a visual picture of spending vs earnings over time
Filter Transactions	Switch between All, Income Only, or Expense Only views
Delete Transactions	Remove any record with a single click
Currency Switcher	Change symbol to USD, EUR, GBP, INR, or JPY
Dark Mode	Toggle dark/light theme; preference is remembered on next visit
Profile Settings	Save your display name and preferred currency
Reset All Data	Wipe everything and start completely fresh
Login / Logout	Simple session check - no real authentication required

2. How the App is Structured

The entire application lives in a single HTML file. There are no separate JavaScript or CSS files to manage. Styles are written inside a style block and all the logic sits in a script block at the bottom. This makes the project very easy to share, deploy, or hand to someone else.

Sections Inside the Page

Single-File Approach

One file means one thing to deploy. Drop it on any web server and it works - nothing to compile, nothing to install, no configuration needed.

Part	What lives inside it
Styles block	All CSS - layout, colors, dark mode variables, responsive rules
Navbar	Logo, navigation links, Add Transaction button, Logout button
Dashboard section	4 summary cards, cash flow chart, filter buttons, transaction table
Settings section	Name input, currency picker, dark mode toggle, reset button
Modal popup	Add Transaction form - hidden by default, shown on button click
Script block	All JavaScript - saving, rendering, charting, filtering, theming

Page Navigation - No Routing Needed

The Dashboard and Settings are both present in the HTML at the same time. JavaScript simply hides one and shows the other when a nav link is clicked. This avoids full page reloads and keeps everything snappy.

3. Approach - How to Build It

The sections below describe the thinking and approach for building each part of FinTrack Pro. Rather than copying code, read each step to understand what it is doing and why - then build it your own way.

Step 1 - Start with the HTML Base

1

Set up the page foundation

Create the HTML document. Link a clean font (Inter from Google Fonts works well). Set a viewport meta tag so it scales correctly on phones. Load Chart.js from a CDN at the bottom of the page.

Things to think about:

- Give the page a meaningful browser tab title
- The viewport meta tag is essential for mobile - without it, the layout will look zoomed out on phones
- Chart.js can be loaded from a CDN so no installation is needed

Step 2 - Build the Navbar

2

Create the top navigation bar

The navbar sits fixed at the top. It has the app name/logo on the left, two navigation links in the centre, and action buttons on the right. Clicking a nav link switches which section is visible - it does not navigate to a new page.

Things to think about:

- Use flexbox with space-between to push the logo left and buttons right
- The nav links call a `showPage()` function rather than linking to separate pages
- A subtle bottom shadow makes the navbar feel elevated above the content

Step 3 - Summary Cards

3

Four stat cards at the top of the dashboard

These cards show Current Balance, Total Income, Total Expense, and Total Transactions. Each number needs a unique identifier so JavaScript can reach in and update it whenever the data changes.

Things to think about:

- Use CSS Grid with 4 equal columns - it collapses to 2 or 1 column on smaller screens
- Color-code meaningfully: green for income, red for expense, indigo for balance
- The numbers should be large and bold - they are the most important thing on screen

Step 4 - Cash Flow Chart

4

A bar chart built with Chart.js

Place a canvas element where the chart should appear. JavaScript creates the chart inside it. Whenever data changes the old chart is destroyed and a new one is drawn with the latest data.

Things to think about:

- Always destroy the existing chart before creating a new one - otherwise they stack on top of each other
- Use green bars for income and red bars for expense so the colors match the rest of the UI
- Transaction dates become the labels along the bottom axis

Step 5 - Transaction Table

5

A table that lists every transaction

The table has columns for Date, Description, Category, Amount, and a Delete action. The table body starts empty. JavaScript fills it in by reading from storage whenever the page loads or data changes.

Things to think about:

- Add three filter buttons above the table: All Types, Income Only, Expense Only
- Each filter button re-renders the table with a different subset of transactions
- Color the amount column green for income rows and red for expense rows
- The delete button in each row needs to know which transaction to remove - use the unique ID for this

Step 6 - Add Transaction Modal

6

A popup form for adding transactions

A modal is a div hidden on the page. Clicking Add Transaction reveals it. The form collects five pieces of information: type (income or expense), description, amount, date, and category. On save, it validates everything before storing anything.

Things to think about:

- The form should validate all fields before saving - show a clear message if something is missing
- Clicking the close button or clicking outside the modal should dismiss it
- After saving, always close the modal and refresh the dashboard immediately
- Categories: Food & Dining, Shopping, Recharge & Bills, Petrol & Auto, Utilities, Salary, Entertainment, Other

Step 7 - Settings Page

7

Profile and preference controls

The settings page is hidden by default, revealed when the user clicks Settings in the nav. It contains a name input, a currency dropdown, a dark mode toggle, and a reset button.

Things to think about:

- The name and currency preference should persist - save them to storage so they are there on next visit
- Changing the currency should immediately refresh all displayed amounts with the new symbol - no data changes, only the display
- Dark mode should be applied on page load if the user previously enabled it
- The reset button should ask for confirmation before wiping everything

Step 8 - JavaScript Logic

8

Wire all the pieces together

All the interactivity sits in one script block. It reads from and writes to browser storage and updates the visible content on every data change. The key principle: every time data changes, one master function refreshes the cards, the table, and the chart all at once.

Core functions to think through:

- Load transactions - read the saved list from storage, return an empty array if nothing is saved yet
- Save transactions - write the updated list back to storage as JSON
- Add a transaction - validate inputs, build a transaction object with a unique ID, save it, close the modal, refresh everything
- Delete a transaction - filter out the matching ID, save the remaining list, refresh everything
- Calculate totals - loop through the list and separately sum income and expense amounts
- Render the table - clear the existing rows, apply any active filter, build a new row for each transaction
- Update the cards - call calculate totals and push the results into the card elements
- Render the chart - destroy the old chart, create a new one with current transaction data
- Master refresh function - calls update cards, render table, and render chart together in one go
- Toggle dark mode - add or remove the dark class on the body element, save the preference
- Show page - hide the current section, reveal the requested one

4. Data Flow - How Everything Connects

Every interaction follows the same pattern: the user does something, the data in storage changes, and the master refresh function updates the entire UI. Once you understand this loop, the whole app makes sense.

The Golden Rule

Always call the master refresh function after any change to data. It updates the cards, redraws the table, and refreshes the chart in one go - nothing goes out of sync.

Adding a Transaction

1. User clicks Add Transaction in the navbar
2. The modal appears on screen
3. User fills in all five fields and clicks Save
4. The save function validates every field - stops and alerts if anything is empty
5. A transaction object is built with a unique ID, the type, description, amount, date, and category
6. The object is added to the list and the list is written back to storage
7. The modal closes
8. Master refresh runs - cards update, table rebuilds with the new row, chart redraws

Deleting a Transaction

9. User clicks Delete on a table row
10. The delete function receives that transaction's unique ID
11. The list is filtered to remove the matching entry
12. The filtered list is saved back to storage
13. Master refresh runs - everything updates with that transaction gone

Changing Currency

14. User picks a different currency in Settings
15. The new currency code is saved to storage
16. Master refresh runs
17. Every amount on screen is formatted using the new currency symbol
18. No transaction data is changed - only how the numbers are displayed

Dark Mode

19. User toggles the dark mode switch in Settings
20. The dark class is added to the body element
21. CSS variables defined for dark mode take over - colors change everywhere automatically

22. The preference is saved to storage

23. Next time the page loads, it checks storage and applies dark mode before anything renders

5. Key Design Decisions

These are the choices that shape how FinTrack Pro is built and why they make sense for this kind of project.

Browser Storage Instead of a Database

Trade-off

Browser storage is instant to use and needs zero backend. The downside is data lives only in that one browser - it will not sync to another device. For a personal or learning project this is completely fine.

Browser storage lets you save, read, and delete data with three simple lines of JavaScript. A real database needs accounts, server configuration, and backend code. For a finance tracker used by one person on one machine, browser storage is the right call.

No JavaScript Framework

Frameworks like React or Vue are powerful but they add a lot: you need a build tool, a package manager, and knowledge of how the framework works before you can write a single line of actual app logic. Plain JavaScript means opening one file in a browser and everything just works.

- Easier to read and debug - what you write is what runs in the browser
- No version conflicts, no dependency updates, no build failures
- Anyone can open the file and understand it without knowing a specific framework

Unique IDs Using Timestamps

Every transaction needs a unique ID so the delete function knows exactly which one to remove. Using the current timestamp in milliseconds is a simple and reliable approach. Since no two transactions can be saved at the exact same millisecond, collisions are not a real concern for a single-user app.

Chart.js from a CDN

Chart.js is one of the most well-documented charting libraries available. Loading it from a CDN means adding one script tag - no installation, no imports, no build step. It handles all the canvas drawing complexity so you can focus on passing it the right data.

CSS Variables for Theming

CSS variables let you define colors once and use them everywhere. When dark mode is toggled, only the variable values change and every element that references them updates automatically. This is far simpler than writing separate light and dark versions of every style rule.

- Define background, card, and text colors as variables in the default (light) state
- Override those same variables under the dark class on the body

- Toggle dark mode by adding or removing that one class - the rest happens automatically

Single Page, Multiple Sections

Rather than creating separate pages for Dashboard and Settings, both sections live in the same document. JavaScript hides one and shows the other when the user clicks a nav link. This avoids page reloads, feels faster, and keeps the project as a single deployable file.

6. Styling Approach

The app uses clean, modern styling that feels professional without being complicated. All styles are written in one block at the top of the document.

Typography

- Use a clean sans-serif font - Inter is a great choice, loaded for free from Google Fonts
- Body text should be comfortable to read at around 14 to 16 pixels
- The numbers in the summary cards should be noticeably large - they are the main thing users look at
- Headings are bold and clearly larger than body text

Color Palette

Color	Role	Used for
Indigo	Primary / Accent	Navbar background, buttons, heading accents
Green	Income	Income amounts in cards, income bars in chart, positive values
Red	Expense	Expense amounts in cards, expense bars in chart, delete buttons
White / Light gray	Cards (light mode)	Card and section backgrounds when in light mode
Deep navy / Slate	Dark mode	Page and card backgrounds when dark mode is active

Layout

- Navbar: full width, fixed to the top, uses flexbox to space out its elements
- Cards: CSS Grid with 4 equal columns on desktop, 2 on tablet, 1 on mobile
- Table: full width, subtle alternating row shading helps readability
- Modal: fixed position in the center of the screen, sits on top of a dark overlay behind it

Dark Mode Implementation

The cleanest approach is CSS variables. Define all colors as variables once, then override them inside a dark rule on the body. Toggle that rule with JavaScript when the user switches the theme.

- Light mode: light backgrounds, dark text
- Dark mode: deep navy backgrounds, light text - the same variables, different values
- Every color in the stylesheet uses a variable - nothing is hard-coded twice

7. Running and Deploying

Running Locally

Since this is a self-contained HTML file with no build step, running it takes seconds:

24. Write all your HTML, CSS, and JavaScript into one document
25. Open it in any modern browser - Chrome, Firefox, Edge, Safari
26. You can double-click the file or drag it directly into the browser window
27. The app is running immediately

Using a Local Server

If anything behaves unexpectedly when opening the file directly, try using VS Code's Live Server extension. It serves the file on localhost which more closely matches how it will behave when deployed.

Deploying to Vercel

Vercel is the platform this project uses. It is free for personal projects and deploys automatically from a connected code repository.

28. Create a free account at vercel.com
29. Push your project to a GitHub repository
30. On Vercel, create a new project and connect your GitHub account
31. Select the repository - Vercel detects it is a static site automatically
32. It deploys and gives you a live public URL

Auto-Deploy on Every Push

Once connected, every time you push a change to your repository Vercel rebuilds and updates the live site automatically. No manual steps after the first setup.

Other Free Hosting Options

- GitHub Pages - host directly from your GitHub repository, completely free
- Netlify - very similar to Vercel, also supports drag-and-drop deployment
- Cloudflare Pages - fast global delivery, also free for personal projects

8. What to Build Next

Once the core app is working, here are ideas for extending it. They are roughly ordered from simplest to most involved.

Beginner-Level Ideas

- Export transactions as a CSV file that opens in Excel or Google Sheets
- A text search bar above the table to filter transactions by description
- Emoji icons next to each category name to make the table easier to scan
- A small badge on the nav showing how many transactions are saved

Intermediate Ideas

- Group transactions by month and show a monthly summary view
- Let users set a spending limit per category and highlight when they exceed it
- Mark a transaction as recurring so it appears again automatically each month
- A print-friendly view or a way to generate a simple PDF report

Advanced Ideas

- Connect to a cloud database so data syncs across devices and browsers
- Add real user accounts with email and password authentication
- Turn the app into a Progressive Web App so it can be installed on a phone like a native app
- Send transaction data to an AI API and let it generate spending insights or suggestions

Good First Extension

Start with the CSV export. It teaches you how to trigger file downloads in the browser using pure JavaScript and gives users something genuinely useful right away.